Rust Tooling and Cargo

Based on ECM2433 rust_01_tooling slides. Covers why Rust exists, the Cargo build system, key commands, project structure, and the cargo fmt and cargo clippy quality tools.

What You Need To Know

1. Why Rust?

Rust targets the class of bugs that are common in C and C++ programs and are difficult to detect at compile time in those languages.

- **Use-after-free**: accessing memory that has already been freed.
- **Double-free**: freeing the same allocation more than once.
- **Buffer overflow**: writing beyond the end of an allocated region.

Rust's ownership system catches most of these at compile time. Array bounds are checked at runtime (panics instead of silent corruption). Unlike a garbage-collected language, Rust achieves this without a runtime GC, so performance stays close to C and C++.

`rustc` is the Rust compiler. In normal projects you usually run it through `cargo`, which coordinates compiling, dependencies, tests, and project layout.

2. Cargo: build system and package manager

Cargo replaces the role of Makefiles, CMake, and manual dependency management in one tool. It handles compiling your code, fetching dependencies, running tests, and more.

- `cargo new project_name` — create a new project.
- `cargo build` — compile to `target/debug/`.
- `cargo check` — type-check without producing a binary. Faster than build when you only want to catch errors.
- `cargo run` — compile (if needed) and run.

```
$ cargo new hello_rust   # creates the project
$ cd hello_rust
$ cargo run              # compiles and runs src/main.rs
   Compiling hello_rust v0.1.0
    Finished dev [unoptimized + debuginfo] target(s)
    Running `target/debug/hello_rust`
Hello, world!
```

3. Project structure

`cargo new` creates a minimal project with three important files:

- `src/main.rs` — the entry point. Contains the `main` function.
- `Cargo.toml` — project configuration you edit: name, version, edition, and dependencies.
- `Cargo.lock` — auto-generated record of the exact dependency versions in use. Do not edit by hand.

```
# Cargo.toml (you edit this)
[package]
name = "hello_rust"
version = "0.1.0"
edition = "2021"

[dependencies]
# add crates here
```

4. Code quality: cargo fmt and cargo clippy

The slides state: run both before submitting work.

- `cargo fmt` — reformats your code to the standard Rust style automatically. No arguments needed.
- `cargo clippy` — a linter that catches mistakes and suggests improvements beyond what the compiler reports.

Example: the compiler accepts the following function, but clippy flags it as unnecessarily verbose and suggests a simpler form.

```
// clippy warning: this if/else can be simplified
fn is_even(n: i32) -> bool {
    if n % 2 == 0 { true } else { false }
```

```
}

// clippy suggestion: return the boolean expression directly
fn is_even(n: i32) -> bool {
    n % 2 == 0
}
```

5. Offline documentation and IDE support

- `rustup doc` — opens locally installed Rust documentation in a browser.
- `rustup doc --book` — opens the Rust Book (the primary reference) offline.
- **rust-analyzer** — the Rust language server. Gives autocomplete, inline errors, go-to-definition, and type hints inside VS Code. Install the rust-analyzer extension to use it.

Practice Questions

1 Name three classes of memory bug that Rust's design aims to prevent.

2 What two roles does Cargo combine that in a C project would require separate tools?

3 What files and directories does `cargo new hello_rust` create?

4 What is the difference between `Cargo.toml` and `Cargo.lock`? Which one should you edit by hand?

5 What is the difference between `cargo build` and `cargo check`? When would you prefer check over build?

6 What does `cargo run` do if the source has not changed since the last successful build?

7 What does `cargo fmt` do to your code?

Question 8 — state what cargo clippy would say about this function:

```
fn is_even(n: i32) -> bool {
    if n % 2 == 0 { true } else { false }
}
```

8 State what cargo clippy would flag in the function above and write the corrected version.

9 How do you open the Rust Book without an internet connection?

10 A student installs the rust-analyzer VS Code extension. Name two benefits they get while editing a Rust file.

Mark Scheme

Attempt all questions before checking.

1. Use-after-free, double-free, buffer overflow (any three of these).
2. Build system (replaces Makefiles / CMake) and package manager (handles fetching and versioning dependencies).
3. `src/main.rs` (entry point with a hello-world main function), `Cargo.toml` (project config), `Cargo.lock` (auto-generated dependency lock), and a `.git/` directory if Git is available.
4. Cargo.toml is the human-edited project configuration: name, version, edition, and declared dependencies. Cargo.lock is auto-generated by Cargo and records the exact versions actually used — do not edit it by hand.
5. cargo build compiles and produces a binary in target/debug/. cargo check only type-checks the code without producing a binary — it is faster and useful when you want to catch errors quickly during development without waiting for a full compile.
6. It skips recompilation and runs the existing binary directly — Cargo detects that nothing has changed.
7. It reformats the source files in place to match the standard Rust style conventions. It makes no logic changes.
8. Clippy warns that the if/else returning true/false is redundant. The corrected version is: `fn is_even(n: i32) -> bool { n % 2 == 0 }`
9. `rustup doc --book`
10. Any two of: autocomplete, inline compiler errors as you type, go-to-definition, type hints / type inference display.

Useful Commands

```
cargo new my_project     # create project
cargo build              # compile
cargo check              # type-check only (faster)
cargo run                # compile + run
cargo fmt                # auto-format code
cargo clippy             # lint and suggest improvements
rustup doc               # open local docs
rustup doc --book        # open the Rust Book offline
```